

BEYOND THE PIXEL

A Major Project Research Paper Submitted in Partial Fulfilment of the Requirement for the
Award of Degree of

BACHELOR OF TECHNOLOGY

(COMPUTER SCIENCE AND ENGINEERING)

To



UTTARAKHAND TECHNICAL UNIVERSITY, DEHRADUN

Submitted by

Aditya Kumar (220410170003)

Parth Panwar (220410170019)

Sahil Singh Chauhan (220410170024)

Aaryan Raj Sinha (220410170001)

Submitted to

Ms. Swati Kashyap

Assistant Professor



**Shivalik College of Engineering
Dehradun, Uttarakhand**

UTU ROLL NUMBER: -

220410170001

220410170003

220410170019

220410170024

Branch: - Computer Science & Engineering (AI-ML)

Batch: - 2022-2026

Proposed Topic: - Beyond the Pixel: A Full-Stack Deepfake Detection System Using Multi-Model Ensemble and Face-Centered Analysis

1. ABSTRACT

Deepfake detection remains a practical challenge because manipulated media often preserves realistic global appearance while introducing subtle local artifacts around facial regions. This repository implements a full-stack deepfake detection system that combines a DFDC-trained image classifier, face detection with MTCNN, frame-based video analysis, and a FastAPI + React interface for interactive inference. The backend uses a PyTorch-based DeepFakeClassifier with an EfficientNet-B7 encoder and processes each input by detecting and cropping a face before classification. For videos, the system samples frames, scores the detected faces independently, and aggregates the results into a final prediction. The frontend provides a simple operator-facing workflow for uploading images and videos and viewing confidence scores. This paper describes the system design, model pipeline, training configuration, deployment architecture, and current limitations based on the code in the repository.

.

2. INTRODUCTION

Synthetic media generation tools have lowered the cost of creating convincing forged images and videos. As a result, there is increasing demand for lightweight forensic systems that can flag suspicious content without requiring specialized workflows. The project in this repository, Beyond the Pixel, addresses that need by integrating a deep learning detector into a usable web application.

The design follows a practical assumption used by many face-forensics systems: a forged image or video is most likely to expose its artifacts in the face region. Instead of classifying the entire frame directly, the system first extracts a face crop and then applies a deepfake classifier to that crop. This approach is used for both still images and sampled video frames.

.

3. LITERATURE SURVEY

The following key works provide the technical and scientific foundation for this system:

- **DFDC (Deepfake Detection Challenge) Dataset:** The primary training resource for the classifier. It provides a large-scale, diverse collection of deepfake and real video clips spanning multiple generation techniques, making it well-suited for training a generalized detector.
- **Selim Seferbekov's DFDC Solution:** The `dfdc_deepfake_challenge` subtree in this repository originates from a top-performing DFDC competition solution. It supplies the `DeepFakeClassifier` architecture, training pipeline, configuration files, and pretrained `EfficientNet-B7` weights used at runtime.
- **EfficientNet: Rethinking Model Scaling for CNNs (Tan & Le, ICML 2019):** Introduced `EfficientNet`, a family of scalable convolutional networks. The B7 variant used here achieves strong image classification accuracy while remaining practical for inference, making it an effective backbone for the deepfake classifier.
- **FaceForensics++ (Rossler et al., ICCV 2019):** Established the face-centric deepfake detection paradigm that this system follows — crop the face region first, then classify. This benchmark demonstrated that face-centered classifiers consistently outperform whole-frame approaches.
- **facenet-pytorch / MTCNN:** PyTorch implementation of multi-task cascaded convolutional networks for reliable face detection and alignment, used as the mandatory first step before classifier inference.
- **Albumentations:** A fast and flexible image augmentation library used for deterministic inference preprocessing — resize to 380x380 and normalize pixel values.

4. METHODOLOGY

4.1 FACE-CENTERED CLASSIFICATION

The detector is intentionally face-centric. In `backend/models/image_detector.py`, the image is opened with PIL, passed through MTCNN, and converted into a face tensor before classification. If no face is detected, the system returns a failure response rather than making a weak guess from the background. This design choice reduces the search space for the classifier and aligns the model input with the DFDC training regime, which is centered on cropped face regions.

4.2 IMAGE DETECTOR

The image detector performs the following steps:

1. Load RGB image.
2. Detect a single face using MTCNN.
3. Convert the face tensor to NumPy format.
4. Resize to 380 x 380 with Albumentations.
5. Normalize pixel values to [0, 1].
6. Run the PyTorch EfficientNet-B7 classifier.
7. Apply sigmoid to obtain a deepfake score in [0, 1].

Score thresholds: LIKELY REAL (score < 0.40) | SUSPICIOUS (0.40 – 0.60) | LIKELY FAKE (score >= 0.60)

4.3 VIDEO DETECTOR

The video detector in `backend/models/video_detector.py` uses a frame-sampling strategy:

1. Open the video with OpenCV.
2. Iterate through frames.
3. Keep every fifth frame.
4. Save the sampled frame to disk.
5. Reuse the image detector on the frame.
6. Average all valid frame scores.
7. Assign the final clip-level label using the same thresholds as the image path.

This approach is simple and computationally efficient compared with dense frame evaluation. It trades some temporal precision for lower latency and a narrower implementation surface.

4.4 END-TO-END PIPELINE

The full runtime inference flow is:

1. User uploads an image or video via the React frontend.
2. Backend stores the file locally in `backend/uploads/`.
3. The detector extracts a face with MTCNN.

4. The face is resized to 380 x 380 and normalized.
5. The DeepFakeClassifier (EfficientNet-B7) generates a raw score.
6. The score is mapped to a human-readable label and confidence indicator.
7. The frontend displays the result.

5. IMPLEMENTATION

5.1 MODEL ARCHITECTURE

The repository uses a DeepFakeClassifier with an EfficientNet-B7 encoder (tf_efficientnet_b7_ns). The training configuration in dfdc_deepfake_challenge/configs/b7.json specifies:

- **Input size:** 380 x 380
- **Batch size:** 12
- **Optimizer:** SGD with momentum 0.9 and weight decay 1e-4
- **Learning rate:** 0.01 with polynomial schedule over 40 epochs
- **Mixed precision:** Enabled
- **Loss function:** Binary cross-entropy

The classifier checkpoint is loaded at runtime in backend/models/image_detector.py. The code removes 'module.' prefixes to support weights saved through DataParallel.

5.1 PREPROCESSING AND NORMALIZATION

For inference, the detector uses narrow and deterministic preprocessing: face detection with MTCNN, resize to 380 x 380, and normalize by dividing by 255.0. The model is therefore exposed to the same semantic object class at inference time that it was trained on: a cropped face region.

5.3 BACKEND (FastAPI)

The backend is a FastAPI application defined in backend/app.py. It exposes two endpoints:

- **POST /detect-image:** Accepts image uploads, writes the file to disk, calls the image detector, and returns a JSON payload with the prediction and confidence score.
- **POST /detect-video:** Accepts video uploads, writes the file to disk, calls the video detector, and returns a JSON payload with the clip-level prediction and score.

The backend enables CORS for http://localhost:5173 to match the Vite-based frontend development server. Uploaded files are stored in backend/uploads/ and temporary frames in backend/outputs/.

5.4 FRONTEND (React + TypeScript)

The frontend is implemented in React 18 with TypeScript (Vite) and provides two independent detection panels - one for image uploads and one for video uploads. The UI workflow is: upload a file, run analysis, and inspect the confidence score bar and prediction label. The interface displays loading states during inference and connection error states when the API is unavailable, lowering the operational barrier for non-technical users.

5.4 INFERENCE SEMANTICS

The system outputs a continuous score rather than a binary prediction, mapping it to three qualitative labels. This avoids overclaiming certainty on borderline inputs, gives operators a meaningful confidence signal, and supports practical human review workflows. The video path uses score averaging across sampled frames, so the model behaves as a clip-level estimator rather than a frame-by-frame alarm system.

6. RESULT

Based on the system architecture and implementation, the following key results and observations can be noted:

- **Face-centered accuracy:** Restricting classification to MTCNN-cropped face regions reduces noisy background signals and aligns inference with the DFDC training distribution, yielding reliable predictions on frontal-face inputs.
- **Three-tier confidence scoring:** The LIKELY REAL / SUSPICIOUS / LIKELY FAKE threshold system provides actionable output without overstating certainty on ambiguous cases.
- **Efficient video analysis:** Frame-score averaging across every fifth frame produces stable clip-level estimates without the computational cost of dense per-frame evaluation.
- **Practical usability:** The React frontend makes the system accessible to non-technical operators, significantly reducing the barrier for forensic review.
- **Known accuracy caveat:** The current repository does not include benchmark evaluation scripts, so there are no reproducible accuracy numbers from the code alone. Performance is expected to align with the DFDC competition results of the underlying Seferbekov EfficientNet-B7 model on similar data distributions.
- **Known limitations:** Requires a clearly detectable frontal face (returns error otherwise); CPU inference is slow and GPU is recommended for production; frame averaging may miss localized or transient manipulations.

7. FUTURE SCOPE

Several improvements would strengthen the system in future iterations:

- Replace mean frame aggregation with confidence-weighted aggregation or a top-k frame policy to better capture localized or transient manipulations.
- Add GPU-aware batching for faster video inference, enabling real-time or near-real-time processing.
- Support webcam and stream-based real-time deepfake detection.
- Add explainability outputs such as attention heatmaps or facial artifact localization to improve operator trust and model interpretability.
- Introduce evaluation scripts and benchmark logging to report measurable performance on DFDC-style validation sets.
- Containerize the full stack using Docker for reproducible deployment across environments.
- Add automatic cleanup of uploaded files in backend/uploads/ for production-ready deployment.
- Implement model confidence calibration and uncertainty quantification for more reliable score interpretation.
- Add support for additional face detection backends such as RetinaFace for better coverage of profile and partially occluded faces.
- Cloud deployment on AWS or Google Cloud with a performance benchmarking dashboard.

8. REFERENCE

- Radford, A., et al. CLIP: Learning Transferable Visual Models From Natural Language Supervision. OpenAI, 2021.
- DeepfakeBench: A Comprehensive Benchmark for AI-Generated and Manipulated Face Detection.
- Rossler, A., et al. FaceForensics++: Learning to Detect Manipulated Facial Images. ICCV, 2019.
- Zhang, K., et al. facenet-pytorch: MTCNN Face Detection and Recognition. PyPI / GitHub.
- Buslaev, A., et al. Albumentations: Fast and Flexible Image Augmentation Library. Information, 2020.
- Tan, M., & Le, Q. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML, 2019.
- Dolhansky, B., et al. The DeepFake Detection Challenge (DFDC) Dataset. arXiv, 2020.
- Seferbekov, S. DFDC Solution Repository and Associated Training Pipeline. GitHub, 2020.